

Kotlin – One for all

Let's get you started with Kotlin, Android's latest pick.

Agnibha Bose | feedback@digit.in

Nowadays a lot of programming languages have surfaced and it's natural for you find it difficult to develop in just a single language. For folks like us who struggle with multiple programming languages for different scenarios, Kotlin can be a saviour.

SO WHAT IS KOTLIN?

Kotlin is a strongly typed programming language, that runs on JVM but can also be compiled by any compatible JS Compiler or LLVM (Low Level Virtual Machine) Compiler. So ideally you can write once and typically run anywhere where these three kinds of compilers are available.

Kotlin has been developed by the JetBrains team, the same team that has created a plethora of IDEs like IntelliJ IDEA, PyCharm, WebStorm etc.

Kotlin is also the third language which is fully supported on android after Java and C++.

WHY DO I NEED TO USE KOTLIN? I DON'T WANT TO LEARN ANOTHER NEW SET OF SYNTAX....

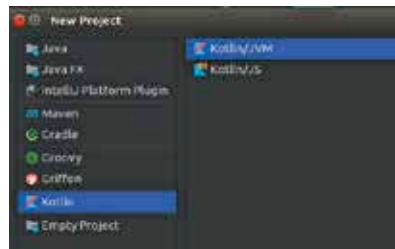
To start of with Kotlin is lot like Java, in essence the way Kotlin is written. Kotlin is very concise and very simple to read. If you are starting fresh then this is ideal because you only need to learn one language instead of three.

Kotlin also provides some nifty features to include android

activity or html tag builders within the language.

Now in this article we're going to use IntelliJ as the IDE because it is by far the most supported one. Starting with a Kotlin Project is very easy in IntelliJ. You'll be asked to select a coding way using Kotlin. We will start of with Kotlin + JVM.

Now we start of with the coding part. Code execution starts from the main function similar to that of Java.



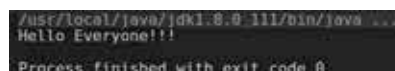
First Create a file named helloworld.kt, we will then create a basic hello world function. Some thing which goes like this:

```
fun main(args: Array<String>){
    println("Hello Everyone!!")
}
```

Now running a Kotlin File is very easy and can be done in IntelliJ quite easily. Just Right-Click on the file and click on Run.

After the compilation is done, the kotlin compiler creates an out folder and a .class file for the JVM to run.

After successful execution of the code the output should look like this.



Now coming to some basic syntax of the code.

1. DEFINING PACKAGES

Packages are defined similar to java . Which means, we'll be defining them at the top of the program.

```
package src
import java.util.*
```

Note:

A number of packages are imported itself by Kotlin. Those are

- kotlin.*
- kotlin.annotation.*
- kotlin.collections.*
- kotlin.comparisons.* (since 1.1)
- kotlin.io.*
- kotlin.ranges.*
- kotlin.sequences.*
- kotlin.text.*

Also, other JVM and JS Packages are imported.

Import statements are also Similar to Java or in the new ES6 declaration. You may also import packages by using the following code:

```
import foo.*
```

Or

```
import foo.Bar
import bar.Bar as bBar
```

2. DEFINING FUNCTIONS

Functions can be defined in many ways in the Kotlin syntax.

Example Function of two variables with a return type:

```
fun sum(a: Int, b: Int): Int{
    return a + b
}
```

Here, fun is the keyword to define a function. 'a' and 'b' are defined as two arguments of the function and a : followed by a dataType is defining the type of the arguments. Here, the last ':' Int' is used to define the return type of the function.

The same function can be defined in this way:

```
fun sum(a: Int, b: Int) = a + b
```

This is similar to something that we are used to with ES6 where the return type is inferred.

Similar to ES6, we can also define default arguments:

```
fun sum(a: Int, b: Int = -1) = a + b
```